Term Project — Details 1

Operating Systems • Inha University in Tashkent • Fall 2026 — the first half of the project

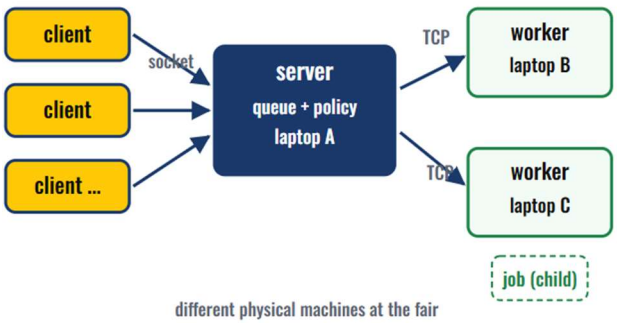
COVERS	Processes and signals · the shell and CPU scheduling · pipes and dup2 · FIFOs · message queues, shared memory and mutual exclusion · semaphores · sockets · concurrent servers · synchronisation
ISSUED	October 14
INSTRUCTOR	Yusupov Jalolliddin
ASSISTANT	Shoakbar Murodov
UPLOAD	One upload per team on eClass, at the end of the semester. The exact date comes with Details 2.
FAIR	In the December exam weeks, on your own laptops. The schedule comes with Details 2.
WEIGHT	20% of the course
WORK	In your registered team of 5-6

There are **no intermediate deliveries and no checks** before the upload: you decide when your team starts and how fast it goes. Everything is collected once, at the end of the semester, and judged at the fair. **Term Project Details 2** arrives after the midterm with the additions for the second half of the course, your team's *twist cards*, the grading and the dates; this document is what you can build today.

1. What you are building

Every team builds the same thing: a small **job runner**. A **job** is a shell command somebody wants executed on another computer — `sort big.txt`, `gcc -o game game.c`, `sleep 30`. Three programs cooperate:

- the **client** — a short command-line program: it submits a job, asks what happened to it, cancels it, or fetches its output;
- the **server** — one process, always running: it accepts clients, keeps the **queue** of jobs, and decides which job runs next and on which worker. That decision is your **scheduling policy**;
- the **worker** — a process on **another machine**: it connects to the server, receives a job, runs it as a child process, captures its output, enforces its time limit, and reports the exit status honestly.



Clients talk to the server over a socket; the server talks to workers over TCP, because workers live on other laptops. The whole course so far is in this picture: fork, exec, wait and signals, a scheduling policy, a pipe that captures a child's output, a queue under a semaphore, sockets and a server for many clients.

In pseudo-code, the whole system is this; the rest of this document is detail.

```
Client:
    connect to server
    send("submit", cmd, timeout)
    id = receive()
    print(id)
    -- later --
    send("status", id)
    print(receive())
```

```
Server:
    loop {
        c = accept()
        serve c without
          blocking the others
        on submit: queue.add(job)
        when a worker is free:
          job = policy.next(queue)
          send job to that worker
        on report: job.state = …
    }
```

```
Worker:
    connect to server
    loop {
        job = receive()
        pid = fork()
        child: exec(job.cmd),
               stdout → pipe
        parent: start timer,
          collect output,
          wait(pid) → status
        send report(id, status, output)
    }
```

2. The rules that carry a zero

At the fair, the server and at least one worker run on two different physical laptops. A one-laptop demo scores zero for the whole project.

- The delivered system runs on **Ubuntu Linux**. Server, worker and client are written in **C**; shell is allowed only for build glue and test scripts.
- A component whose declared owner cannot explain it at the fair scores zero **for that member**. The team is not punished for one passenger, and the passenger is not carried by the team.
- Work a student cannot explain or modify live, at the fair, is graded as absent.
- Until the fair, **two virtual machines on one laptop are fine** for development. Bridged networking, which lets two laptops see each other over Wi-Fi, will be shown in class.

3. Your team

The system splits into **five components**, one owner each; a team of six declares the sixth component in the table. Anyone may touch the code, but the owner is the person the examiner questions about it at the fair and the one who must change that code live.

Component	What the owner personally delivers, and is asked about
Protocol and client	The wire format (message names, fields, framing, error replies), the client program. "A client dies half way through a submit. Where does your read loop handle that?"
Scheduler and policy	The job queue, the chosen policy, the dispatch decision, a trace that proves the policy behaves as claimed. *"Switch to priority now. What starves, and what stops it?"*
Worker and limits	The worker: fork, exec, wait, output capture, the timeout kill, honest exit status, no zombies. *"Which line turns a timeout kill into KILLED, not DONE 0?"*
Durability and recovery	In Phase 1: the job table and its states, cancel, what happens when a worker says goodbye. Phase 2 adds the rest. *"I cancel a running job. Which processes notice, and how?"*
Operations and report	The Makefile, the repository and its history, the report. In a team of five, also everything in the next row. "Show me a fresh clone building, and where the trace in your report came from."
Testing and load (the sixth component, teams of six)	The script that starts eight clients at once, a self-check script that runs the twelve rows of §5.4 unattended, the zombie and descriptor checks. "Run the test that proves no descriptor leaks. Which check would have caught last week's bug?"

- **Write the owner of every component and the team leader into README.md in the first commit** of your repository, next to your team number. Nobody checks it before the end of the semester, but your git history shows when you actually started, and that the work was not prepared one day before the deadline.
- **The team leader is a role, not a component.** Every team names one: the leader coordinates, runs the weekly meeting, merges the branches and keeps the report moving, and still owns one of the components like everyone else. There is no component called "management".
- A team of six declares six components. A team of five declares five, the operations owner taking testing and load as well**. A team that is smaller than five, for whatever reason, still declares all five components**: the members divide them among themselves, a member may own more than one, and each member answers the questions on every component they own at the fair. That is the team's own responsibility; the project does not get smaller with the team.
- Every job id your server issues starts with your team number: team 12 issues T12-1, T12-2, ...

> **Git, if it is new to you.** Your code lives in a git repository on GitHub or GitLab (free accounts), and every member pushes their own commits. Take ten minutes to study git if you are not familiar with it: the official tutorial at `https://git-scm.com/docs/gittutorial`, and `https://learngitbranching.js.org` to practise. Five commands are enough for this project: `clone`, `add`, `commit`, `push`, `pull`. Commit small and often; your history is part of what we read.

4. What the system must do

Operation	What happens
`submit`	The client sends a command, a **timeout in seconds** and, optionally, a priority; the server answers with a new job id. The job is **QUEUED**.
`status`	The server answers with the job's state: **QUEUED**, **RUNNING**, **DONE** with its exit code, **KILLED** with the reason, or **CANCELLED**.
`cancel`	A QUEUED job leaves the queue. A RUNNING job is killed by its worker. Either way the state becomes **CANCELLED**. A finished job cannot be cancelled.
`fetch`	Everything the job printed, standard output **and** standard error, in the order the job wrote them (send both into the same pipe), at least the first 64 KiB. Above your own limit, cut it off and state the limit in the report. Only for finished jobs.

A job runs under **fork and exec** on a worker, **always through the shell**: the child execs /bin/sh with -c and the command string, so that exit 3, pipes and quotes behave exactly as they do in a terminal. Do not split the command into words yourself. Output is captured through a pipe or a file on the worker and kept where fetch can find it, on the server or on the worker: your design.

> **Exit status, honestly.** `waitpid` gives you a status integer. `WIFEXITED(st)` is true if the child called `exit`; then `WEXITSTATUS(st)` is its exit code (0 for success, 3 for `exit 3`). `WIFSIGNALED(st)` is true if a signal killed it; then `WTERMSIG(st)` is the signal number (9 for SIGKILL). A job that was killed is reported as **KILLED**, never as **DONE 0**. Exit 137 is not success.

4.2 The time limit

Every job carries a timeout chosen by the client. The worker starts a timer with the job; if the job is still running when it fires, the worker kills it and the state becomes **KILLED TIMEOUT**. The kill must take **everything the job started**: `sh -c "sleep 100"` is two processes, and killing only the shell leaves sleep alive.

> **Process groups.** In the child, right after `fork`, call `setpgid(0, 0)`: the job becomes its own process group, and everything it starts inherits the group. Later `kill(-pid, SIGKILL)`, with a **minus** sign, kills the whole group. Use the same call for `cancel`. For the timer, `alarm()` or a helper process that sleeps and then kills: your choice.

The worker must remember **why** it killed a job, because waitpid cannot tell a timeout from a cancel from a kill by somebody else.
Timeout → **KILLED TIMEOUT**; cancel → **CANCELLED**; anything else → **KILLED SIGNAL** with the number.

4.3 The scheduling policy

A worker runs at most **K jobs at once** (K is given on the worker's command line, default 1) and **tells the server its K in its first message**; the server keeps a count of free slots per worker. Whenever a worker has a free slot, the server picks the next QUEUED job: **that choice is the policy**. **FIFO**, the oldest job first, is the minimum and where everyone starts. Beyond it, choose one of these or design your own, and name it in the report:

- **Shortest job first** — the job with the smallest timeout first, the timeout being your only estimate of length. Long jobs starve; aging fixes that.
- **Priority with aging** — highest priority first. 9 is the highest; the default 0 is the lowest, so a job that says nothing never jumps the queue. A job that has waited long enough gains priority, so nobody waits forever.

- **Multi-level feedback queue** — several queues, demotion after a quantum. Preemptive, so the server must tell the worker to **pause** and **resume** a job, because only the worker can signal it (SIGSTOP and SIGCONT on the job's process group); a paused job stays on its worker, frees its slot, and still reports RUNNING. Hard.

The assistant's script never tests the policy; your **report's trace** and the scheduler owner's questions at the fair do. A team that stays with FIFO has nothing to say there and nothing to defend. The optional priority on `submit` is an integer 0–9; FIFO and shortest job first ignore it.

4.4 Many clients at once, and clean hygiene

- The server serves **at least 8 clients at once**. A client that connects and says nothing for ten seconds must not stop another client from submitting. A server that forks per client, or keeps a pool, does this; the one-at-a-time server does not.
- **Zero zombies.** Every child is reaped: the SIGCHLD reaper with the `waitpid(-1, …, WNOHANG)` loop from Homework 1, or a `waitpid` on each job you know.
- **Zero leaked file descriptors.** Close both pipe ends where they are not needed (the closing rule for pipes) and every connection you finish with. We count open files before and after a hundred jobs; the number must not grow.
- **Clean shutdown on SIGTERM.** Handle the signal; do not die from it. The server prints a line, closes its listening socket and exits within two seconds; a new server on the same port starts immediately afterwards. A worker on SIGTERM kills its running jobs, reaps them, reports them as **KILLED SIGNAL 9**, and only then exits, also within two seconds. If the server cannot be reached at that moment, the worker still kills, reaps and exits; the report is best effort, never a reason to hang. A worker that vanishes *without* warning is Phase 2's subject.

> **Counting open files.** `ls /proc/<pid>/fd | wc -l` lists the open descriptors of process `pid` and counts them. Run it on your server and worker before and after a loop of a hundred `echo hi` jobs.

4.5 Transport

Client to server: **TCP**. Server to worker: **TCP, and the worker connects to the server**, so the server needs no list of workers and your laptops' firewalls stay closed. The wire format is yours. One line per message, ending in `\n`, read in a loop, because one `write` is not one `read`. The one exception is the answer to `fetch`: it has many lines and may end without a newline, so it needs a length prefix or an end marker. `net.h` from Homework 2 may be used unchanged.

5. The command-line contract

The assistant checks the core behaviour with **one script, run on every team's repository**, which is only possible if every team's programs are started and driven the same way. The contract below is therefore exact; the inside of your programs and your protocol are free. **Deviating from it fails every check the script cannot reach.**

5.1 Building (we will cover the `make` soon)

```
$ git clone <your repository> team12 && cd team12
$ make                        # in the root of the repository, produces three executables:
$ ls server worker client     # exactly these names, in the root
```

`make` must succeed on a fresh Ubuntu 22.04 or 24.04 with `build-essential` and nothing else; `make clean` removes what it built. Warnings are allowed, errors are not. Suggested layout: `src/` for the three programs and their shared headers, `tests/` for your scripts, and `README.md` with the team number, the team leader and the owner of every declared component. Do **not** create folders named `server`, `worker` or `client` in the root: the executables need those names, and `gcc -o server` fails if a folder called `server` is already there.

make, if it is new to you. `make` reads a file named `Makefile` in the current folder and runs the commands needed to build whatever is out of date. The smallest one that satisfies this contract:

```
all: server worker client
server: src/server.c
        gcc -Wall -o server src/server.c
worker: src/worker.c
        gcc -Wall -o worker src/worker.c
client: src/client.c
        gcc -Wall -o client src/client.c
clean:
        rm -f server worker client
```

The indented lines begin with a **tab**, not spaces, or make refuses the file. Add `-pthread` where you use semaphores, and list every `.c` file a program is built from.

5.2 Starting the programs

Command	Behaviour
`./server <port>`	Listens on TCP `<port>` on all interfaces. Prints `server: listening on port <port>`, then one line per event (client connected, job accepted, job sent, job finished). Runs until SIGTERM.
`./worker <server-host> <server-port> [K]`	Connects to the server, prints `worker: connected to <host>:<port>, K=<K>`, runs up to K jobs at once (default 1). Several workers may connect. Reconnecting after a server restart is welcome, not required in Phase 1.
`./client <server-host> <server-port> <verb> …`	One request, one answer, exit. The four verbs are below. A client that cannot connect prints `client: cannot connect to <host>:<port>` to standard error and exits with **3**.

5.3 The four client verbs, exactly

Verb and arguments	Prints to standard output	Exit code
`submit "<command>" <timeout> [priority]`	The new job id alone on one line, e.g. `T12-7`. `<timeout>` is in whole seconds, at least 1.	0
`status <id>`	One line: `T12-7 QUEUED` · `T12-7 RUNNING` · `T12-7 DONE 0` (the exit code) · `T12-7 KILLED TIMEOUT` · `T12-7 KILLED SIGNAL 9` (the signal number) · `T12-7 CANCELLED`	0
`cancel <id>`	`T12-7 CANCELLED`. If the job had already finished: `T12-7 ALREADY FINISHED`.	0 · 1
`fetch <id>`	The job's captured output, byte for byte, nothing added. If the job has not finished: nothing on standard output, `T12-7 NOT FINISHED` on standard error.	0 · 1
any verb with an id the server never issued	`T12-99 UNKNOWN`	2

The script compares the **first line** of the client's standard output and the exit code: extra lines after the first are harmless, extra words on the first line are not. "Within 2 s" below is generous; the script waits twice that.

5.4 What the script does, and what it expects

Run this yourselves before you upload. The script runs **everything on one machine over `127.0.0.1`**; two machines are checked live at the fair. `$C` stands for `./client 127.0.0.1 5000`.

#	Step	Expected
1	`make`; `./server 5000 &`; `./worker 127.0.0.1 5000 1 &`	Three executables; the two start lines are printed.
2	`$C submit "echo hello" 10` → id; 2 s later `$C status <id>`; `$C fetch <id>`	An id; `<id> DONE 0`; `hello`
3	`$C submit "exit 3" 10`; `$C submit "ls /nonexistent" 10`; status and fetch of both	`DONE 3`; `DONE 2` and the fetched output contains `No such file`
4	`$C submit "sleep 100" 2`; 4 s later `$C status <id>`; then `pgrep -xf "sleep 100"`	`KILLED TIMEOUT`; `pgrep` finds nothing
5	`$C submit "sleep 30" 60`; 1 s later status; `$C cancel <id>`; status; `pgrep -xf "sleep 30"`	`RUNNING`; `CANCELLED`; `CANCELLED`; nothing running
6	With K = 1: three `sleep 3` jobs submitted back to back; `pgrep -xfc "sleep 3"` every half second until all three are DONE	The count never exceeds 1; all three `DONE 0` within 12 s
7	Eight clients started in the same second, each `submit "sleep 2" 10`	Eight different ids within 2 s; all eight `DONE 0` within 25 s

8	`nc 127.0.0.1 5000` connects and types nothing; meanwhile `$C submit "echo x" 10`	The submit gets its id within 2 s
9	`$C status T12-99999`	`T12-99999 UNKNOWN`, exit code 2
10	`ps -o pid,ppid,stat,comm --ppid <worker pid>` after all of the above	No process in state `Z`
11	`ls /proc/<pid>/fd | wc -l` for server and worker, before and after 100 `echo hi` jobs	The same number before and after, for both
12	`$C submit "sleep 30" 60`; 1 s later `kill -TERM <worker pid>`; 2 s later `$C status <id>` and `pgrep -xf "sleep 30"`; then `kill -TERM <server pid>` and `./server 5000` again at once	`KILLED SIGNAL 9`; nothing running; the worker has exited with no zombies; the server exits within 2 s and the new one starts without `Address already in use`

> **pgrep** finds running processes by name: `pgrep sleep` prints the pids of every running `sleep`; `-x -f` matches the **whole command line exactly**, so `pgrep -xf "sleep 100"` finds that job but neither the shell that started it (its line is `sh -c sleep 100`) nor the script's own waits; `-c` counts instead of listing. **nc** (netcat) connects to a port and waits for you to type, a perfect silent client. Both are on every Ubuntu.

Each row is pass or fail. A row the script cannot run because the contract was not followed fails.

6. The report and the fair

Grading will be explained in the second phase document, Details 2, together with the twist cards and the dates. What is already certain: the twelve rows of §5.4 are checked by script, the report is read, and at the fair you are asked about your own component.

6.1 The report

One PDF, **four pages maximum**, uploaded with the code. The template comes with Details 2. It asks for the design with one diagram, the protocol, the policy with a trace that proves it, the Phase 2 additions, the five owners, and **one real failure** you had, with your own logs and what you changed. Written from what you built, not what you hoped to build.

6.2 The fair

- **Five minutes of demo** on your own laptops, server and worker on two physical machines. The examiner may interrupt with a request — kill a worker mid-job, start eight clients at once, change K while running — and nothing prepared in advance counts.
- **Five to ten minutes of questions**, at the code, to the members the examiner chooses, on the components they own. The grade is what you can explain and change live.
- The schedule and the booking come with Details 2.

7. Where to start

No deadlines, only an order: the one the lectures taught, and every step is something you did once in a homework.

1. **The protocol on one page.** Message names, fields, one line per message. The protocol owner drafts it, everyone says yes. Nothing else starts before this page exists.
2. **Hello across two machines.** One message over TCP, one reply, with `net.h` from Homework 2.
3. **Server skeleton.** Accept a client, parse one `submit`, answer an id, serve several clients at once.
4. **Worker.** Connect, announce K, receive a command, fork and exec it with its output in a pipe, wait, report.
5. **`status` and `fetch`.** One job now makes the whole round trip: the vertical slice. Celebrate.
6. **The queue and the policy.** K slots per worker is the *limit* pattern; the queue is the pass shelf.
7. **Timeout and cancel.** Process groups, `alarm`, `kill(-pid, …)`.
8. **Eight clients and hygiene.** The reaper, the closing discipline, the SIGTERM handler; then run §5.4 yourselves.

Three design rules that will save you in November

- All job records live in **one table** in the server, and **every state change goes through one function**. Nobody writes `job->state = …` anywhere else.
- A job record can be written out as **one line of text** and read back. Make that function exist even if nothing uses it yet.
- The worker starts a job through **one function**, `run_job()` or whatever you name it, and nothing else calls `fork`.

Upload checklist

1. One upload per team on eClass: the repository URL **and** a ZIP of the same commit.
2. `make` in the root builds `server`, `worker`, `client` on a fresh Ubuntu.
3. `README.md` names the team number, the team leader and the owner of every component, and has since the first commit.
4. §5.4 passes on your own machine, with the commands exactly as written.
5. `report.pdf` is in the root, four pages or fewer, on the Details 2 template.